SYSTEM DESIGN CHEAT SHEET

patterns · scaling · rate limiting · latency · failure reconciliation · archetypes · estimations

DATA PATTERNS

PATTERN	WHEN TO USE	TRADE-OFFS
DATA `CQRS`	Read/write patterns diverge. Scale reads independently. Denormalized read view needed. Postgres (write) + Redis/ES (read)	X Eventual consistency — read lags writes *V Independent scaling per model*
DATA `Event Sourcing`	Audit trail required. Reconstruct state at any point. Multiple consumers react differently. Kafka, Postgres append-only	X Current state = expensive replay; use snapshots *V Full history, time travel, decoupled consumers*
DATA `CDC`	Sync 3+ stores (DB + cache + search). React to DB changes without app code changes. Debezium + Kafka + Postgres WAL	X Overkill for <3 consumers *V No dual-write problem*
DATA `Saga`	Multi-step txn across services. No distributed ACID. Each step is compensatable. Temporal, AWS Step Functions	X Eventual consistency; compensations are hard *V No distributed locking needed*
DATA `Outbox`	Guaranteed at-least-once delivery. Solve dual-write crash problem. Outbox table + relay → Kafka/SQS	X Extra table + relay process *V Atomic DB write + event in one txn*

RELIABILITY PATTERNS

PATTERN	WHEN TO USE	TRADE-OFFS
RELY `Circuit Breaker`	Calling slow/external deps. Want fail-fast over timeout. Resilience4j, Hystrix — Closed → Open → Half-open	X Threshold tuning; fallback must be designed *V Protects system under dep failure*
RELY `Bulkhead`	Batch jobs could starve user requests. Critical paths need guaranteed capacity. Separate thread pools, K8s limits	X Resource fragmentation; pool sizing tricky *V Blast radius containment*
RELY `Retry + Backoff + Jitter`	Any network call to external dep. Transient failures expected. tenacity (Python), SDK built-ins	X Latency on failure; set max retry ceiling *V Jitter prevents thundering herd*
RELY `Idempotency`	Any retryable write (payments, bookings). At-least-once queue consumers. UUID key → DB/Redis with TTL	X Storage for key-result map; TTL calibration *V Retry-safe, no duplicate side effects*

COMMUNICATION PATTERNS

PATTERN	WHEN TO USE	TRADE-OFFS
COMM `Fan-out on Write`	One event → many recipients. Low-latency reads required. Hybrid: write for normal users, read for celebrities	X Write amplification — O(followers) per post *V O(1) reads — feed pre-built per user*

SCALING STRATEGIES

STRATEGY	WHEN TO USE	TRADE-OFFS
SCALE `Horizontal Scale`	Stateless API servers, CPU/memory bottleneck. ALB + ECS/K8s + auto-scaling; session → Redis	X Must be stateless; more instances = more ops *V Linearly scalable*
SCALE `Caching (Redis)`	DB reads are bottleneck. Data read >> written. cache-aside · write-through · write-behind	X Cache invalidation is hard; stale = TTL window *V Highest ROI; 10-100x DB load reduction*
SCALE `Read Replicas`	Read traffic overwhelms primary. Low lag acceptable. Postgres streaming replication, PgBouncer	X Replication lag — stale reads possible *V Simple; 3-5x read headroom*
SCALE `Sharding`	Write throughput or data volume exceeds single node. After replicas+cache exhausted. Vitess, Citus, consistent hashing	X Cross-shard joins expensive; rebalancing painful *V Near-unlimited horizontal write scale*
SCALE `Async Queue`	Work exceeds latency budget. Need retry. Absorb spikes. Kafka · SQS · Celery+Redis	X Harder to debug; always add DLQ *V Decouples producer/consumer; absorbs spikes*
SCALE `Indexes + Denorm`	Slow queries before adding infra. Check EXPLAIN first. B-tree, composite, partial, GiST/GIN (Postgres)	X Writes slower; data duplication with denorm *V Cheapest perf win — always check first*

RATE LIMITING ALGORITHMS

ALGORITHM	MEMORY	BURST?	WHEN TO USE / TRADE-OFF
RL `Fixed Window`	O(1) — 1 counter	Yes (exploit)	Internal/low-stakes only. X Boundary burst: 2x limit straddles window edge
RL `Sliding Log`	O(limit)	No	Auth, payments — precision over memory. X Memory explodes at high volume *V Perfect accuracy; Redis sorted set*
RL `Sliding Counter`	O(1) — 2 counters	No	General-purpose high scale. Cloudflare uses this. X ~0.4% estimation error *V Best accuracy/memory tradeoff*
RL `Token Bucket *`	O(1) — 2 values	Controlled	Default for user-facing APIs. Stripe, GitHub, OpenAI use this. X Needs atomic update (Lua script in Redis) *V Tune burst ceiling + refill rate independently*

LATENCY NUMBERS EVERY ENGINEER SHOULD KNOW

L1 cache reference	~0.5 ns
L2 cache reference	~7 ns
RAM access	~100 ns
SSD random read	~16 µs
HDD seek	~2–10 ms
Redis GET (local)	<1 ms
Same datacenter round trip	~0.5 ms
Same region (cross-AZ)	~1–3 ms
Postgres simple query	~1–5 ms
Postgres cold (no index)	100ms–sec
Cross-region (US–EU)	~60–100 ms
Cross-region (US–Asia)	~150–200 ms

FAILURE MODE RECONCILIATION

MODE	WHEN TO USE	TRADE-OFFS
PROAC `Outbox + Idempotency`	Must never drift (payments, ledger). Prevent inconsistency at write time. Postgres txn + outbox table + UUID key	X Extra table + relay; atomic update complexity *V Drift structurally impossible; retry-safe*
PROAC `Optimistic Locking`	Concurrent writes could race. Low-contention resources. version column + WHERE version = :v in UPDATE	X Retry on conflict; not for high-contention *V No lock overhead; lightweight*
REACT `DLQ Replay`	Failed async job left data in inconsistent state. Message dropped after max retries. SQS DLQ + replay tool; log trace ID + payload	X Manual review needed for financial messages *V No silent data loss; replayable on fix*
REACT `Reconciliation Job`	Derived store silently drifted from source of truth. Incremental diff: query updated_at > last_run; emit repair event on mismatch	X Lag = job interval; doesn't catch real-time drift *V Low overhead; catches silent failures*
REACT `Saga Compensation`	Multi-step saga failed mid-flight. Need to undo committed steps. Compensation events per step; saga log for state	X Compensations must also be idempotent; complex *V System self-heals without manual surgery*
PASS `TTL + Cache-aside`	Staleness acceptable within bounded window (dashboards, scores). Redis TTL; on miss read from DB and repopulate	X Stale up to TTL; never use for balances *V Zero ops; self-corrects on expiry*
	Derived store corrupted or fully out of sync. Audit log is source of truth.	X Rebuild takes time; needs partitioned replay

PATTERN	WHEN TO USE	TRADE-OFFS
COMM Push (WS/ SSE)	Real-time updates. Sub-second latency. Live scores, collab editing. WebSocket (bidirectional), SSE (server->client)	X Server tracks connected clients; client must be online *V Lowest latency; no wasted polling*
COMM API Gateway	Multiple services behind one endpoint. Centralize auth, rate limiting, tracing. AWS API GW, Kong, Nginx, Envoy	X Single point of failure; can become bottleneck *V Cross-cutting concerns centralized*

ALGORITHM	MEMORY	BURST?	WHEN TO USE / TRADE-OFF
RL Leaky Bucket	O(queue size)	No (smooth)	Protecting downstream with hard throughput ceiling. X Adds latency; queue management overhead

BOTTLENECK → STRATEGY

BOTTLENECK	FIRST REACH FOR	THEN CONSIDER
API CPU/memory	Horizontal scale + LB	Auto-scaling groups
DB reads	Read replicas + Redis	Denorm, indexes
DB writes	Connection pool, batching	Sharding, CQRS
Hot path latency	Redis / in-process cache	Async, pre-compute
Traffic spikes	Message queue	Rate limiting, auto-scale
Slow queries	Indexes (EXPLAIN first)	Denorm, read replicas
Cross-region latency	CDN + geo replicas	Edge compute

MODE	WHEN TO USE	TRADE-OFFS
PASS Audit Log Rebuild	Drop + replay event log; snapshots avoid full replay	*V Guaranteed correctness; no manual data surgery*

SYSTEM ARCHETYPES — HEAR X, THINK Y

SYSTEM TYPE	CORE PROBLEMS TO ADDRESS IMMEDIATELY
Reservation / booking	Double-booking prevention (optimistic lock or atomic check-and-set), hold TTL + state machine, idempotency
Banking / ledger	Double-entry bookkeeping, idempotency key (non-negotiable), strong consistency (ACID, not eventual)
Notification system	Fan-out model (write vs. read), multi-channel routing, at-least-once + idempotency per channel
Search / typeahead	Index freshness (CDC async), ranking model, trie or Redis sorted set for autocomplete
Social feed	Fan-out on write (normal) vs. read (celebrity), Redis sorted set per user, cursor pagination
Chat / messaging	WebSocket for real-time, message ordering (sequence number), write-heavy storage (Cassandra)
Ride-sharing / geo	Redis GEOADD for driver locations, GEOSEARCH for nearby, trip state machine, WebSocket for live map
URL shortener	Hash collision handling, 301 (browser caches) vs 302 (analytics), cache-heavy (reads >> writes)
Web crawler / pipeline	URL frontier (priority queue), deduplication (Bloom filter), politeness (rate limit per domain)

BACK-OF-ENVELOPE RULES OF THUMB

SYSTEM / RESOURCE	THROUGHPUT CEILING	NOTES
Single Postgres (simple reads)	~10K QPS	Add replicas beyond this
Single Redis instance	~100K–1M QPS	In-memory; almost never the bottleneck
Single Kafka partition	~10–50 MB/s	Add partitions to scale
Single API server (light)	~1K–10K RPS	Stateless; scale horizontally
1 Gbps network link	~125 MB/s	CDN if bandwidth exceeds ~10 GB/s
QPS from DAU	DAU × actions/day ÷ 86,400	Peak = avg × 2–3x
Storage estimate	record size × records × replication	Add 20–50% index overhead; replicate 2–3x
RAM vs SSD vs HDD	RAM 1000x faster than SSD	SSD 100x faster than HDD — justify caching with this